

Compléments d'Algorithmique

Chapitre 3 : Programmation orientée objet

Hadrien Mélot
Service d'Algorithmique
Département d'Informatique
Faculté des Sciences



Chapitre 3. Programmation orientée objet

- 1 Rappels
- 2 Héritage et polymorphisme
- 3 Créer ses propres structures dynamiques
- 4 Concept de listes chaînées
- 5 Implémentation d'une classe de base
- 6 Piles et Files

Rappels

Rappels

La Programmation Orientée Objet (POO) a été introduite dans le dernier chapitre du cours d'*Algorithmique et programmation impérative*.

Pour se rafraîchir la mémoire, nous allons construire une classe pour modéliser des joueurs.

Pour rappel, toute classe devrait avoir

- un constructeur `__init__` qui initialise les attributs de tout objet de cette classe,
- une méthode `__str__` qui retourne une représentation conviviale de l'objet pour les utilisateurs.

Rappels : constructeur et méthode `__str__`

Définition de classe

```
class Joueur:
    def __init__(self, pseudo, niveau = 1):
        self.pseudo = pseudo
        self.niveau = niveau
        self.xp = 0
        self.sante = 100

    def __str__(self):
        etat = ""
        if self.sante > 80:
            etat = "en pleine forme"
        elif self.sante > 50:
            etat = "fatigué"
        elif self.sante > 0:
            etat = "dans un sale état"
        else:
            etat = "Mort (RIP)"
        return f"{self.pseudo} (Niveau: {self.niveau}) : {etat}"
```

Rappels : constructeur et méthode `__str__`

Exemple d'utilisation

```
p1 = Joueur("Noob")  
print(p1)  
p2 = Joueur("Pro", 10)  
print(p2)
```

Résultat

```
Noob (Niveau: 1) : en pleine forme  
Pro (Niveau: 10) : en pleine forme
```

Méthode spéciale `__repr__`

Il peut être utile de définir aussi `__repr__` qui fournit une représentation "interne" de l'objet (avec toutes ses variables d'instance), typiquement utilisée par les développeurs.

Nous ajoutons aussi une méthode `name` pour n'avoir que le pseudo.

Définition de classe (suite)

```
class Joueur:
    (...)

    def __repr__(self):
        return f"{self.pseudo} | Niveau: {self.niveau} | \
                XP: {self.xp} | Santé: {self.sante}"

    def name(self):
        return self.pseudo
```

Remarque : le symbole `\` est utilisé ici pour dire que la ligne continue sur la suivante.

Méthode spéciale `__repr__`

Exemple d'utilisation (suite)

```
print(p1)
print(repr(p1))
print(p1.name())
```

Résultat (suite)

```
Noob (Niveau: 1) : en pleine forme
Noob | Niveau: 1 | XP: 0 | Santé: 100
Noob
```

Nous avons maintenant suffisamment de moyens d'afficher nos objets, en fonction des circonstances.

Rappels : méthodes

On définit le comportement d'un objet via des méthodes.

Définition de classe (suite)

```
class Joueur:
    (...)

    def seuil(self):
        return 10 * self.niveau

    def gagner_xp(self, points):
        self.xp += points
        level_up = False
        while self.xp >= self.seuil():
            self.xp -= self.seuil()
            self.niveau += 1
            level_up = True
        return level_up
```

Rappels : méthodes

Exemple d'utilisation (suite)

```
if p1.gagner_xp(50):  
    print(f"{p1.name()} monte de niveau !")  
print(repr(p1))  
if p2.gagner_xp(50):  
    print(f"{p2.name()} monte de niveau !")  
print(repr(p2))
```

Résultat (suite)

```
Noob monte de niveau !  
Noob | Niveau: 3 | XP: 20 | Santé: 100  
Pro | Niveau: 10 | XP: 50 | Santé: 100
```

Rappels : méthodes

Définition de classe (suite)

```
class Joueur:
    (...)

    def degats(self):
        return self.niveau * 2

    def subir_degats(self, degats):
        self.sante -= degats

    def attaquer(self, other):
        degats_infliges = self.degats()
        other.subir_degats(degats_infliges)
        level_up = self.gagner_xp(10)
        return level_up, degats_infliges
```

Rappels : méthodes

Exemple d'utilisation (suite)

```
lvl, dg = p1.attaquer(p2)
print(f"{p1.name()} attaque {p2.name()} et inflige {dg} dégâts !")
if lvl:
    print(f"{p1.name()} monte de niveau !")
lvl, dg = p2.attaquer(p1)
print(f"{p2.name()} attaque {p1.name()} et inflige {dg} dégâts !")
if lvl:
    print(f"{p2.name()} monte de niveau !")
print(f"{p1}\n{p2}")
```

Résultat (suite)

```
Noob attaque Pro et inflige 6 dégâts !
Noob monte de niveau !
Pro attaque Noob et inflige 20 dégâts !
Noob (Niveau: 4) : fatigué
Pro (Niveau: 10) : en pleine forme
```

Rappels : surcharge d'opérateurs

On peut surcharger les opérateurs pour les adapter à nos objets.

Par exemple, surchargeons l'opérateur < pour comparer des joueurs.

Définition de classe (suite)

```
class Joueur:
    (...)

    def __lt__(self, other):
        return (self.niveau, self.xp) < (other.niveau, other.xp)
```

Rappels : surcharge d'opérateurs

Exemple d'utilisation (suite)

```
print("Pro est le meilleur" if p1 < p2 else "Noob est le meilleur")
print(p1.name(), "gagne 500 XP")
print(p1.gagner_xp(500))
print("Pro est le meilleur" if p1 < p2 else "Noob est le meilleur")
print(repr(p1))
print(repr(p2))
```

Résultat (suite)

```
Pro est le meilleur
Noob gagne 500 XP
Noob est le meilleur
Noob | Niveau: 11 | XP: 10 | Santé: 80
Pro | Niveau: 10 | XP: 60 | Santé: 94
```

Rappels : surcharge d'opérateurs

Surchargeons l'opérateur + pour obtenir un nouveau joueur qui est la fusion de deux joueurs.

Définition de classe (suite)

```
class Joueur:
    (...)

    def __add__(self, other):
        return Joueur(f"{self.pseudo}+{other.pseudo}",
                       self.niveau + other.niveau)
```

Rappels : surcharge d'opérateurs

Exemple d'utilisation (suite)

```
p3 = p1 + p2  
print(repr(p3))
```

Résultat (suite)

```
Noob+Pro | Niveau: 21 | XP: 0 | Santé: 100
```

Ce méga joueur a un niveau de 21 et est en pleine santé, mais il n'a pas d'XP. Il ne faut pas exagérer non plus !

Listes des principaux opérateurs

Opérateur	Méthode spéciale	Description
+	<code>__add__</code>	Addition
-	<code>__sub__</code>	Soustraction
*	<code>__mul__</code>	Multiplication
/	<code>__truediv__</code>	Division
//	<code>__floordiv__</code>	Division entière
%	<code>__mod__</code>	Modulo
**	<code>__pow__</code>	Puissance
<	<code>__lt__</code>	Inférieur
<=	<code>__le__</code>	Inférieur ou égal
==	<code>__eq__</code>	Égal
!=	<code>__ne__</code>	Différent
>	<code>__gt__</code>	Supérieur
>=	<code>__ge__</code>	Supérieur ou égal

Listes des principaux opérateurs (suite)

Opérateur	Méthode spéciale	Description
<code>+=</code>	<code>__iadd__</code>	Addition incrémentée
<code>-=</code>	<code>__isub__</code>	Soustraction incrémentée
<code>*=</code>	<code>__imul__</code>	Multiplication incrémentée
<code>/=</code>	<code>__itruediv__</code>	Division incrémentée
<code>[]</code>	<code>__getitem__</code>	Accès à un élément
<code>[] =</code>	<code>__setitem__</code>	Modification d'un élément
<code>len()</code>	<code>__len__</code>	Taille
<code>in</code>	<code>__contains__</code>	Appartenance
<code>hash()</code>	<code>__hash__</code>	Fonction de hachage

Nous verrons d'autres exemples de surcharge par la suite, mais il ne faut pas en abuser : ajoutez-en seulement si ça a du sens.

Héritage et polymorphisme

Héritage

On peut définir une classe qui hérite d'une autre classe.

Définition de sous-classes

```
class Guerrier(Joueur):  
    def __init__(self, pseudo, niveau = 1, force = 10):  
        super().__init__(pseudo, niveau)  
        self.force = force  
  
    def __repr__(self):  
        return super().__repr__() + f" | Force: {self.force}"  
  
    def degats(self):  
        return super().degats() + self.force
```

Héritage

Remarques :

- Toutes les méthodes de la classe parente sont accessibles dans la classe fille : elles sont automatiquement héritées.
- Une méthode redéfinie dans la classe fille surcharge la méthode de la classe parente ;
- Il est cependant possible d'appeler la méthode de la classe parente originale via `super()` ;
- Grâce à `super().__init__` dans le constructeur de la classe fille, les variables d'instance de la classe parente sont héritées.

Héritage

Remarques pour notre classe `Guerrier` :

- Concernant `gagner_xp` : cette méthode n'est pas redéfinie et est héritée par `Guerrier`;
- Concernant `degats` : cette méthode est redéfinie : elle appelle la méthode héritée de `Joueur` et y ajoute la force du guerrier.
- Concernant `attaquer` :
 - ▶ Cette méthode n'est pas redéfinie et est héritée par `Guerrier`;
 - ▶ Cependant, le code d'`attaquer` fait appel à `degats`, qui elle, est redéfinie dans la classe `Guerrier`;
 - ▶ Cela implique que si `self` est un `Guerrier`, la méthode `attaquer` va faire appel à la version redéfinie de `degats` (qui tiendra donc compte de la force).

Héritage

Exemple d'utilisation

```
p1 = Guerrier("Conan", niveau = 5)
print(repr(p1))
p2 = Joueur("Boss", niveau = 35)
lvl, dg = p1.attaquer(p2)
print(f"{p1.name()} attaque {p2.name()} et inflige {dg} dégâts !")
lvl, dg = p2.attaquer(p1)
print(f"{p2.name()} attaque {p1.name()} et inflige {dg} dégâts !")
print(p1)
```

Résultat

```
Conan | Niveau: 5 | XP: 0 | Santé: 100 | Force: 10
Conan attaque Boss et inflige 20 dégâts !
Boss attaque Conan et inflige 70 dégâts !
Conan (Niveau: 5) : dans un sale état
```

Observez l'adaptation des méthodes `__repr__` et `degats` du Guerrier grâce à sa force.

Héritage

Définition de sous-classes (suite)

```
class Mage(Joueur):
    def __init__(self, pseudo, niveau = 1, intelligence = 10):
        super().__init__(pseudo, niveau)
        self.intelligence = intelligence

    def __repr__(self):
        return super().__repr__() + f" | Intelligence: \
            {self.intelligence}"

    def lancer_sort(self, other):
        degats_infliges = self.intelligence
        other.subir_degats(degats_infliges)
        level_up = self.gagner_xp(30)
        return level_up, degats_infliges
```

Les Mages n'ont pas de force, mais ils sont capables de lancer des boules de feu (en plus de l'attaque qui est héritée) !

Héritage

Exemple d'utilisation

```
p1 = Guerrier("Conan", niveau = 5)
p2 = Mage("Gandalf", intelligence = 20)
lvl, dg = p2.lancer_sort(p2)
print(f"{p2.name()} lance une boule de feu sur {p2.name()} \
      et inflige {dg} dégâts !")
if lvl:
    print(f"{p2.name()} monte de niveau !")
print('Zut ! Gandalf s\'est trompé de cible (mais a gagné de 1\'XP) !')
print(p2)
lvl, dg = p2.lancer_sort(p1)
print(f"{p2.name()} lance une boule de feu sur {p1.name()} \
      et inflige {dg} dégâts !")
lvl, dg = p2.attaquer(p1)
print(f"{p2.name()} attaque {p1.name()} et inflige {dg} dégâts !")
print(repr(p1))
print(repr(p2))
```

Héritage

Résultat

Gandalf lance une boule de feu sur Gandalf et inflige 20 dégâts !

Gandalf monte de niveau !

Zut ! Gandalf s'est trompé de cible (mais a gagné de l'XP) !

Gandalf (Niveau: 3) : fatigué

Gandalf lance une boule de feu sur Conan et inflige 20 dégâts !

Gandalf attaque Conan et inflige 8 dégâts !

Conan | Niveau: 5 | XP: 0 | Santé: 72 | Force: 10

Gandalf | Niveau: 4 | XP: 10 | Santé: 80 | Intelligence: 20

Héritage

Que se passe-t-il si on fusionne un mage et un guerrier ?

Exemple

```
p1 = Guerrier("Conan")
p2 = Mage("Gandalf")
p3 = p1 + p2
print(p3)
print(type(p3))
```

Résultat

```
Conan+Gandalf (Niveau: 2) : en pleine forme
<class '__main__.Joueur'>
```

L'opérateur + fonctionne (Guerrier et Mage héritent de Joueur), mais l'opérateur retourne un Joueur (puisqu'il est défini dans cette classe).

Polymorphisme

Le **polymorphisme** est un concept de POO qui permet d'utiliser une même interface (méthode ou fonction) pour différents types d'objets : celle-ci peut s'adapter en fonction de l'objet.

Par exemple, la méthode `degats` est disponible pour les Mages et les Guerriers, mais

- celle des Mages est la méthode héritée de `Joueur`,
- celle des Guerriers est redéfinie pour tenir compte de sa force.

Le comportement s'adapte donc en fonction du type de l'objet. C'est un des concepts les plus puissants de la POO.

Polymorphisme

Le fait qu'une fonction ou un opérateur puisse agir sur des objets de types différents est un autre exemple du polymorphisme.

Exemple

```
print(max(7, 99))
print(1 > 2)
conan = Guerrier("Conan", 3)
gandalf = Mage("Gandalf", 5)
print(max(conan, gandalf))
print(conan < gandalf)
```

Résultat (suite)

```
99
False
Gandalf (Niveau: 5) : en pleine forme
True
```

Créer ses propres structures dynamiques

Pile

Une **pile** est une structure de données permettant les opérations suivantes :

- insérer un élément en haut de la pile ;
- obtenir (et retirer) l'élément du haut de la pile ;
- obtenir le nombre d'éléments dans la pile ;
- afficher le contenu de la pile ;
- etc.



Ce type de structure représente une pile car le dernier élément inséré est le premier élément retiré (**LIFO (Last In First Out)**).

Fréquent en informatique : par ex. pile des fonctions appelées (*f* appelle *g* qui appelle *h* : *h* sera la première fonction terminée, puis on retourne en *g*, etc.).

Image : cherionethingivelearned.blogspot.be

File

Une **file** est une structure de données permettant les opérations suivantes :

- insérer un élément à la fin de la file ;
- obtenir (et retirer) l'élément au début de la file ;
- obtenir le nombre d'éléments ;
- afficher le contenu ;
- etc.



Ce type de structure représente une file car le premier élément inséré est le premier élément retiré (**FIFO (First In First Out)**).

Fréquent en informatique : files de processus à accomplir, de fichiers à traiter, etc.

Image : Jurgen Ziewe / Ikon Images

Créer nos propres piles et files

Nous pourrions utiliser une liste pour simuler une pile ou une file, en l'utilisant de manière adéquate, ou utiliser d'autres classes disponibles dans les librairies Python.

Mais pour illustrer la POO, nous allons créer nos propres classes pour ces structures de données. Cela nous permettra également de découvrir comment on peut créer ses propres structures de données dynamiques.

Pour y arriver, nous allons introduire la notion de [liste chaînée](#). Nous n'en présenterons que le strict minimum. Cette notion sera approfondie dans les cours d'*Algorithmique et programmation orientée objet* et de *Structures de Données I*.

Structures dynamiques

Nos structures de données doivent permettre de stocker un nombre indéfini d'objets et doivent être mutables.

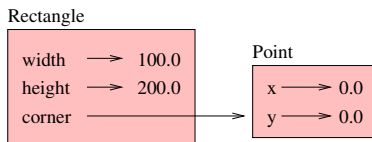
Problème : comment déterminer les attributs des objets de type pile ou file, si nous ne connaissons pas le nombre d'éléments à l'avance, et sans utiliser les structures dynamiques existantes en Python ?

Structures dynamiques

Ce qu'on ne peut pas faire : définir une classe ayant un nombre indéfini d'attributs.

Ce qui est autorisé : un attribut peut être une référence vers un autre objet, de n'importe quel type.

Exemple : un `Rectangle` a un attribut `Point`



Concept de listes chaînées

Listes chaînées

Ce qui est autorisé : un attribut peut être une référence vers un autre objet, de *n'importe quel type*, et donc *y compris un autre objet de la même classe*.

Nous allons définir une classe `Node` dont les objets auront deux attributs :

- `data` : une donnée à stocker
- `next` : une référence vers un autre `Node` s'il y a un élément suivant, ou `None` sinon.

Ensuite, nous lierons ces objets entre eux pour former une *liste chaînée*, via la référence `next`.

Listes chaînées

Définition de classe

```
class Node:
    def __init__(self, data = 0, next = None):
        self.data = data
        self.next = next

    def __str__(self):
        return str(self.data)

    def __repr__(self):
        return repr(self.data) + ' -> ' + repr(self.next)

    def set_next(self, next):
        self.next = next

    def get_next(self):
        return self.next

    def set_data(self, data):
        self.data = data

    def get_data(self):
        return self.data
```

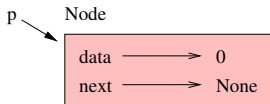
Listes chaînées

Utilisation

```
p = Node()  
print('Donnée dans p:', p)  
print('repr(p):', repr(p))
```

Résultat

```
Donnée dans p: 0  
repr(p): 0 -> None
```



Listes chaînées

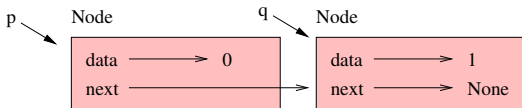
Utilisation (suite)

```
q = Node(1)
p.set_next(q)
print('Donnée dans q :', q)
print('repr(q) :', repr(q))
print('repr(p) :', repr(p))
```

Résultat

```
Donnée dans q : 1
repr(q): 1 -> None
repr(p): 0 -> 1 -> None
```

On peut donc accéder à toutes les données via le premier maillon !



Listes chaînées

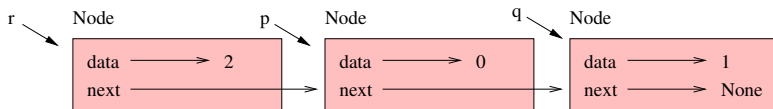
Utilisation (suite)

```
r = Node(2, p)
```

Résultat

Donnée dans r : 2

repr(r): 2 -> 0 -> 1 -> None



Listes chaînées

Si on a une référence vers le premier maillon, on a accès à toutes les données en suivant les références `next`.

Reconstruisons une nouvelle liste chaînée en nous contentant d'une seule référence vers le premier maillon, pour illustrer comment on peut modifier les données et la structure de la chaîne.

Utilisation

```
first = Node(1, Node(2, Node(3)))  
print(repr(first))
```

Résultat

```
1 -> 2 -> 3 -> None
```

Listes chaînées

Liste actuelle : `first` $\rightarrow$ 1 $\rightarrow$ 2 $\rightarrow$ 3 $\rightarrow$ None

Question : comment changer la valeur du deuxième maillon pour y mettre 42 ?

Listes chaînées

Liste actuelle : `first` → 1 → 2 → 3 → None

Question : comment changer la valeur du deuxième maillon pour y mettre 42 ?

Réponse

```
first.get_next().set_data(42)
print(repr(first))
```

Résultat

```
1 -> 42 -> 3 -> None
```

Listes chaînées

Liste actuelle : `first` $\rightarrow$ 1 $\rightarrow$ 42 $\rightarrow$ 3 $\rightarrow$ None

Question : comment ajouter une valeur 7 entre le deuxième et le troisième maillon ?

Listes chaînées

Liste actuelle : `first` → 1 → 42 → 3 → None

Question : comment ajouter une valeur 7 entre le deuxième et le troisième maillon ?

Réponse

```
p = Node(7, first.get_next().get_next())
first.get_next().set_next(p)
print(repr(first))
```

Résultat

```
1 -> 42 -> 7 -> 3 -> None
```

Listes chaînées

Liste actuelle : `first` $\rightarrow$ 1 $\rightarrow$ 42 $\rightarrow$ 7 $\rightarrow$ 3 $\rightarrow$ None

Question : comment supprimer le premier maillon ?

Listes chaînées

Liste actuelle : `first` → 1 → 42 → 7 → 3 → None

Question : comment supprimer le premier maillon ?

Réponse

```
first = first.get_next()  
print(repr(first))
```

Résultat

42 -> 7-> 3 -> None

Implémentation d'une classe de base

Implémentation d'une classe de base

Nous avons donc à notre disposition les éléments nécessaires pour construire nos piles et nos files dynamiques :

- chaque donnée sera stockée dans un `Node` ;
- les `Node` seront liés entre eux pour former une chaîne dont la longueur peut être aussi longue que l'on veut ;
- on peut mettre à jour les données et la structure de la chaîne selon nos besoins.

Nous allons d'abord créer une classe de base `ListeChaine` et ensuite donner le comportement souhaité à deux classes filles pour nos piles et nos files.

Classe de base

Nous désirons une liste simplement chaînée qui supporte les opérations suivantes avec les meilleures complexités possible :

- `__len__()` : pour connaître le nombre d'éléments ;
- `get_first()` : retourne l'élément qui se trouve en tête de la liste chaînée (sans la modifier) ;
- `remove_first()` : retourne l'élément en tête de la liste et le retire de celle-ci ;
- `insert_first(x)` : insère x en tête de la liste.
- `insert_last(x)` : insère x à la fin de la liste.

L'idée est simple : nous pourrons par la suite utiliser cette classe de base pour faire des piles (dont le “haut” sera l'élément au début de la liste) et des files (dont l'élément en tête de la file sera celui au début de la liste chaînée).

Classe de base

Pensez-vous qu'il soit possible de faire toutes les opérations en $O(1)$?

- `__len__()`
- `get_first()`
- `remove_first()`
- `insert_first(x)`
- `insert_last(x)`

Classe de base

Pensez-vous qu'il soit possible de faire toutes les opérations en $O(1)$?

- `__len__()`
- `get_first()`
- `remove_first()`
- `insert_first(x)`
- `insert_last(x)`

Oui, comme on le verra, à condition de conserver

- le nombre d'éléments dans la liste (à cause de `__len__()`);
- une référence vers le dernier maillon de la liste chaînée (à cause de `__insert_last__()`);

Classe de base

Définition de classe

```
class ListeChaine:  
    def __init__(self):  
        self.n = 0  
        self.first = None  
        self.last = None  
  
    def __repr__(self):  
        return repr(self.first)  
  
    def insert_first(self, data):  
        self.first = Node(data, self.first)  
        if self.last is None:  
            self.last = self.first  
        self.n += 1
```

La complexité de `insert_first` est $O(1)$, quelle que soit la taille de la liste chaînée.

Classe de base

Utilisation

```
lc = ListeChaine()
lc.insert_first(1)
lc.insert_first(2)
lc.insert_first(3)
print(repr(lc))
```

Résultat

```
3 -> 2 -> 1 -> None
```

Classe de base

Définition de classe (suite)

```
class ListeChaine :
    (...)
    def get_first(self):
        if self.first is None:
            raise ValueError('Structure is empty')
        return self.first.data

    def remove_first(self):
        if self.first is None:
            raise ValueError('Structure is empty')
        data = self.first.data
        self.first = self.first.next
        if self.first is None:
            self.last = None
        self.n -= 1
        return data

    def __len__(self):
        return self.n
```

Toutes les complexités des opérations requises sont bien en $O(1)$.

Classe de base

Utilisation (suite)

```
print('Valeur en tête de liste:', lc.get_first())
print('Taille de la liste:', len(lc))
removed = lc.remove_first()
print('Valeur retirée:', removed)
print('Valeur en tête de liste:', lc.get_first())
print('Taille de la liste:', len(lc))
print(repr(lc))
```

Résultat

```
Valeur en tête de liste: 3
Taille de la liste: 3
Valeur retirée: 3
Valeur en tête de liste: 2
Taille de la liste: 2
2 -> 1 -> None
```

Classe de base

Comme il s'agit d'une classe de base, nous nous contenterons d'appeler `__repr__` pour implémenter `__str__`. Nous spécialiserons cela plus tard pour nos files et nos piles.

Définition de classe (suite)

```
class ListeChaine:  
    (...)  
  
    def __str__(self):  
        return repr(self)
```

Ici, la complexité est en $O(n)$ car `__repr__` va, depuis le premier maillon, être appelé sur tous les maillons suivants récursivement, mais pour afficher n valeurs, c'est inévitable.

Classe de base

Pour l'insertion en fin de la liste chaînée, nous travaillerons sur la référence `last`.

Définition de classe

```
class ListeChaine:
    (...)

    def insert_last(self, data):
        if self.last is None:
            self.first = self.last = Node(data)
        else:
            self.last.next = Node(data)
            self.last = self.last.next
        self.n += 1
```

La complexité est en $O(1)$ grâce à `last`.

Classe de base

Utilisation

```
lc = ListeChaine()
lc.insert_first("Java")
lc.insert_last("Python")
lc.insert_first("C++")
print(lc)
```

Résultat

```
'C++' -> 'Java' -> 'Python' -> None
```

Classe de base

Surchargeons également l'opérateur `in` (qui aura du sens pour nos piles et nos files).

Définition de classe (suite)

```
class ListeChaine:  
    (...)  
  
    def __contains__(self, data):  
        p = self.first  
        while p is not None:  
            if p.data == data:  
                return True  
            p = p.next  
        return False
```

La complexité de `in` est en $O(n)$, mais c'est inévitable vu la structure utilisée.

Classe de base

Utilisation (suite)

```
if "Python" in lc:  
    print('Python is in the list')
```

Résultat

```
Python is in the list
```

Piles et Files

Piles

Nous désirons les opérations suivantes, qui définissent le comportement d'une pile (LIFO) :

- `__len__()` : pour connaître le nombre d'éléments dans la pile ;
- `get_top()` : retourne l'élément en haut de la pile (sans le retirer) ;
- `pop()` : retourne l'élément en haut de la pile et le retire de celle-ci ;
- `push(x)` : insère `x` en haut de la pile.

En réalité, si nous héritons de `ListeChaine`, tout est déjà implémenté, nous voulons juste donner des noms qui ont du sens par rapport à nos piles.

Piles

Définition de classe

```
class Pile(ListeChaine):  
    def __init__(self):  
        super().__init__()  
  
    def push(self, data):  
        self.insert_first(data)  
  
    def pop(self):  
        return self.remove_first()  
  
    def get_top(self):  
        return self.get_first()
```

Piles

Utilisation

```
stack = Pile()
stack.push(1)
stack.push(2)
stack.push(3)
print(stack)
print('n =', len(stack))
if 2 in stack:
    print('2 is in stack')
x = stack.pop()
print('x =', x)
print('Top:', stack.get_top())
print(stack)
```

Piles

Résultat

```
3 -> 2 -> 1 -> None
```

```
n = 3
```

```
2 is in stack
```

```
x = 3
```

```
Top: 2
```

```
2 -> 1 -> None
```

Piles

Surchargeons la méthode `__str__` pour afficher le contenu de la pile de manière plus conviviale. La variable d'instance `self.max_str` sera ajoutée et gérée par après.

Définition de classe (suite)

```
class Pile(ListeChaine):
    (...)

    def __str__(self):
        res = ''
        p = self.first
        while p is not None:
            res += "|" + str(p.data).center(self.max_str + 2) + "|\n"
            p = p.next
        res += '-' * (self.max_str + 4)
        return res
```

La complexité est en $O(n)$, mais c'est logique puisqu'on affiche n valeurs.

Piles

Pour que notre méthode `__str__` fonctionne, il faut modifier le constructeur et la méthode `push`, pour gérer la variable d'instance `self.max_str`.

Définition de classe (mise à jour)

```
class Pile(ListeChaine):  
    def __init__(self):  
        super().__init__()  
        self.max_str = 0  
  
    def push(self, data):  
        self.insert_first(data)  
        self.max_str = max(self.max_str, len(str(data)))
```

Piles

Utilisation

```
stack = Pile()
stack.push("Mons")
stack.push("Info")
stack.push("Python")
print(stack)
```

Résultat

```
| Python |
|  Info  |
|  Mons  |
-----
```

Files

Passons maintenant à notre file (FIFO). Nous désirons les opérations suivantes :

- `__len__()` : pour connaître le nombre d'éléments dans la file ;
- `get_first()` : retourne l'élément en tête de la file (sans le retirer) ;
- `remove()` : retourne l'élément en tête de la file et le retire de celle-ci ;
- `insert(x)` : insère `x` à la fin de la file.

Nous n'avons plus grand chose à faire grâce à l'héritage !

Files

Définition de classe

```
class File(ListeChaine):
    def __init__(self):
        super().__init__()

    def insert(self, data):
        self.insert_last(data)

    def remove(self):
        return self.remove_first()

    def __str__(self):
        res = ''
        p = self.first
        while p is not None:
            res += str(p.data) + " <- "
            p = p.next
        res = res[:-4] # pour retirer la dernière flèche
        return res
```


Files

Utilisation

```
queue = File()
queue.insert("Alice")
queue.insert("Bob")
queue.insert("Charlie")
print(queue)
first = queue.remove()
print('Go out from queue:', first)
print(queue)
```

Résultat

```
Alice <- Bob <- Charlie
Go out from queue: Alice
Bob <- Charlie
```

Application des piles et des files

Illustration en séance : le Donjon (code disponible sur moodle).